Note 17: Radix Sort

Instructions

Our objective is to study and define the *radix sort* for nonnegative integers.

Background

The *radix sort* exploits the *radix system* (or *positional system*) of numbers. In a radix system, the *radix* (or *base*) k is the number of unique digit values, and each digit is a value in $[0, k-1]$. The value of a number is the sum of its digits, each multiplied by the power of the radix given by its position. A number with digits $x_{d-1}x_{d-2}\cdots x_1x_0$ in radix k has the value

$$\sum_{i=0}^{d-1} x_i k^i.$$

The digit x_0 is the *least significant digit*, and x_{d-1} is the *most significant digit*.

Example: In radix 10, the number `4075` has the digits 4, 0, 7, and 5, and its value is $4 \cdot 10^3 + 0 \cdot 10^2 + 7 \cdot 10^1 + 5 \cdot 10^0$.

The radix sort sorts the elements digit by digit, starting with the least significant digit and ending with the most significant digit. Each pass uses a helper sorting algorithm that must be *stable*, so that the order established by the earlier (less significant) digits is preserved among elements that have equal digits in the current position. The algorithm is:

```
RadixSort(A, d)
  //Traverse from the least to the most significant digit
  for i ← 0 to d, do
    CountSort(A, 10, hash, i)
```

where the modified `CountSort()`, which sorts A by the digit that *hash* retrieves, is defined as:

```
CountSort(A, k, hash, i)
  //Declare an output array B of length A.length and a counter array C of length k
  for j ← 0 to k, do C[j] ← 0
  //Count the occurrences of each digit
  for j ← 0 to A.length, do
    C[hash(A[j], i)] ← C[hash(A[j], i)] + 1
  //Adjust C so that each entry is the number of elements with a digit less than or equal to its index
  for j ← 1 to k, do C[j] ← C[j] + C[j − 1]
  //Place the elements into B, from last to first, to keep the sort stable
  for j ← A.length downto 0, do
    C[hash(A[j − 1], i)] ← C[hash(A[j − 1], i)] − 1
    B[C[hash(A[j − 1], i)]] ← A[j − 1]
  //Copy the result back into A
  for j ← 0 to A.length − 1, do A[j] ← B[j]
```

and, for decimal integers, the function `hash()` is defined as:

`hash(A, i)`: return $\lfloor A/10^i \rfloor \bmod 10$

In general, `hash()` is the function that retrieves the ith digit of a value and converts it to an index in $[0, k-1]$.

The radix sort has a running time of $\Theta(d(n+k))$, where n is the number of elements in the collection, k is the radix, and d is the maximum number of digits of any element of the collection. Each of the d passes is a counting sort that takes $\Theta(n+k)$ time. In general, n is exceeding longer than d and k, which makes the radix sort runs in linear time $\Theta(n)$.

Example

Sorting $A = [329, 457, 657, 839, 436, 720, 355]$ with $d = 3$ and $k = 10$ gives the following array after each pass:

Pass (digit)	**Array after the pass**
$i = 0$ (ones)	$[720, 355, 436, 457, 657, 329, 839]$
$i = 1$ (tens)	$[720, 329, 436, 839, 355, 457, 657]$
$i = 2$ (hundreds)	$[329, 355, 436, 457, 657, 720, 839]$

Note that in the second pass, `720` stays before `329` because both have the tens digit 2 and the first pass placed `720` first.

Tracing

To verify that the algorithm is correct, trace by hand the contents of the array A after each invocation of the stable sorting algorithm (`CountSort()`), for each of the following initial arrays. Use $d = 3$.

1. $A = [415, 743, 747, 893, 706, 537]$
2. $A = [912, 758, 654, 499, 877, 636, 655, 301]$
3. $A = [597, 979, 476, 543, 245]$
4. $A = [486, 239, 403, 797, 790, 178, 129]$
5. $A = [601, 276, 211, 366, 922, 617]$

Coding

Create a header file that includes only the libraries `iostream`, `string`, `sstream`, `cmath`, and `vector`, and defines the following functions:

- `string print(const vector<size_t>& data)`: returns a string of the elements of *data* as a list enclosed in square brackets.
- `size_t hash(size_t x, size_t i)`: returns the ith digit of x, where the digit at index 0 is the least significant digit.
- `void countSort(vector<size_t>& A, size_t i)`: stably sorts A by the ith digit of its elements.
- `void radixSort(vector<size_t>& A, size_t d)`: sorts A using the radix sort on its radix d, and displays A after each invocation of `countSort()`.

Then create a `.cpp` file that tests `radixSort()` on each of the arrays from the Tracing section, using $d = 3$.